Projektiranje autonomnih sustava

Tjedan 5 - uvodno o roboracer kolegiju

Karlo Cirkvenčić, Nino Dragičević

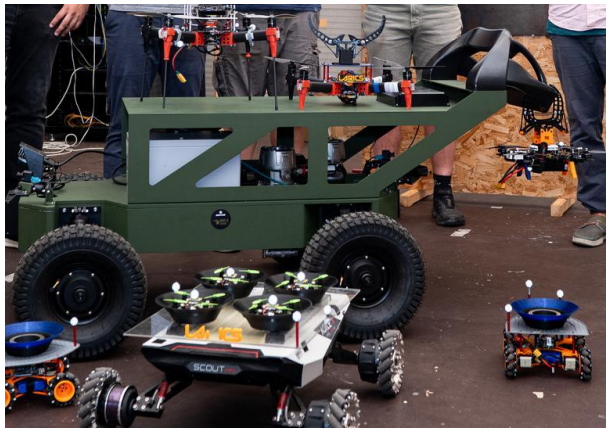
5.11.2026

Što je RoboRacer natjecanje?

- Autonomne utrke vozila u mjerilu 1:10 (bivši naziv F1TENTH)
- Sveučilišni timovi iz cijelog svijeta, utrkuju se na svjetskim robotičarskim konferencijama
- Svaki tim ima isti hardver, razlika je samo u softveru; pobjeđuje algoritam, a ne robot
- Formati utrka: time trial (sam na stazi) i head-to-head (dva auta, pretjecanje)

Cilj natjecanja

- Platforma za učenje percepcije, planiranja i upravljanja na stvarnom robotu
- Cilj kolegija: do kraja semestra vaš kod samostalno vozi auto po stazi
- Zašto je to vrijedno? Slični stack koji se koristi i u industriji





Zašto utrkivanje?

1. Kroz utrku vozilo se dovodi do svojih limita
2. Donošenje odluka na samim limitima vozila
3. Upravljanje vozila pri njegovom limitu

Utrkivanjem probijamo granice hardvera i softvera



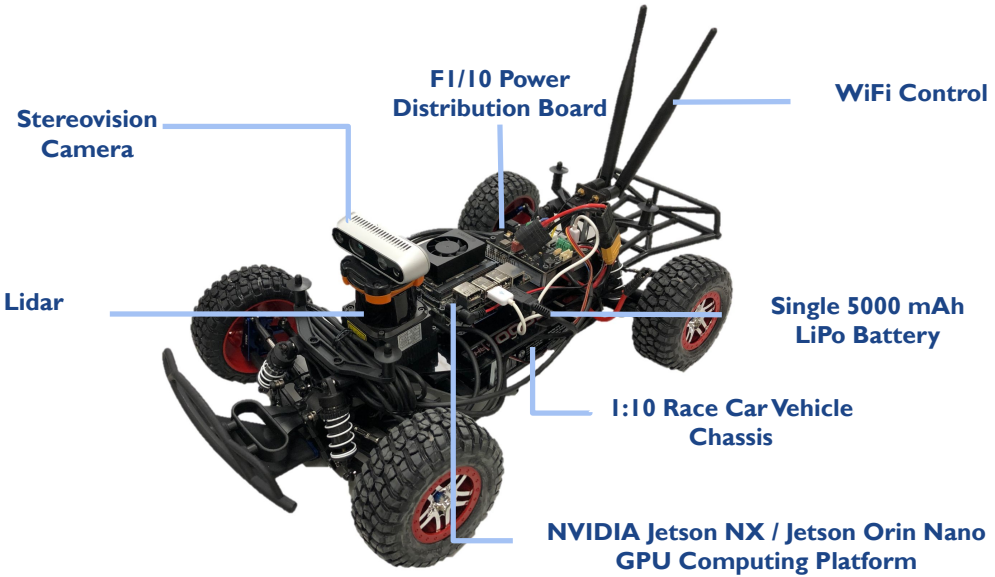




RoboRacer vozilo

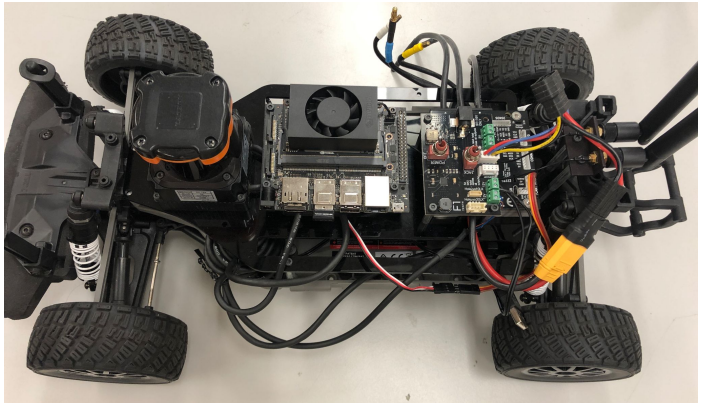
Vozilo podijeljeno:

- Lower chassis
- Upper chassis
- Autonomy sensors



Hardverska arhitektura vozila

- Šasija: RC auto, Ackermann upravljanje (servo za skretanje + BLDC motor)
- Računalo: Jetson Orina Nano - cijela autonomija vrti se na autu
- Senzor: 2D LiDAR (Ethernet) - glavni izvor percepcije
- VESC: upravljanje motorom i servom + odometrija iz motora
- Napajanje: 3s baterija
- Sigurnost: daljinski upravljač s “dead man’s” prekidačem - čovjek uvijek može preuzeti kontrolu

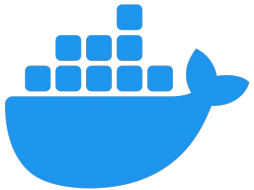


Razvoj algoritama na vozilu

- ROS2 (robot operating system) framework
  - verzija: ubuntu 24.04, ROS2 jazzy
- Git (github) za praćenje verzija koda
- Docker za portabilnost koda







ROS2

- “Framework” za razvoj software-a u robotici
- Glavni dio - komunikacijski sloj (middleware) - temeljen na **DDS**-u (Data Distribution Service)
- Temeljne komponente - **čvorovi** (nodes) koji međusobno komuniciraju
- Open source
- Vrlo dobro podržan sa jako puno gotovih **paketa** (tf2, rviz2, nav2, ros2_control, robot_localization, …)
- 3 glavna načina komunikacije - **teme** (topics), **servisi** (services) i **akcije** (actions)

ROS2 paketi

- Temeljna jedinica organizacije i distribucije u ROS2
- Može biti Python paket ili C++ paket
- Praktički: sadrži sve potrebne datoteke kako bi se pokrenuo kod u ROS2

Kreiranje paketa:

```
ros2 pkg create my_robot_package --build-type ament_cmake
ros2 pkg create my_robot_package --build-type ament_python
```

python paket

```
my_robot_package/
├── package.xml
├── setup.py
├── setup.cfg
├── resource/
│   └── my_robot_package
├── my_robot_package/
│   ├── __init__.py
│   ├── robot_controller.py
│   └── sensor_reader.py
└── launch/
    └── robot.launch.py
```

C++ paket

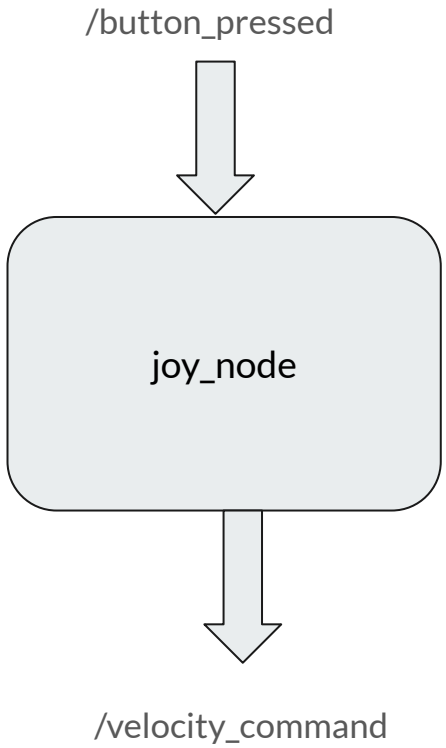
```
my_robot_package/
├── CMakeLists.txt
├── package.xml
├── include/
│   └── my_robot_package/
│       ├── robot_controller.hpp
│       └── sensor_reader.hpp
├── src/
│   ├── robot_controller.cpp
│   └── sensor_reader.cpp
└── launch/
    └── robot.launch.py
```

ROS2 čvorovi

- Izvršni proces (executable) - obavlja određeni zadatak unutar ROS2 sustava
- Najmanji gradivni element u softverskoj arhitekturi robota
- Svaki čvor odgovoran je za predodređenu funkciju i komunicira sa ostalim čvorovima unutar ROS2 sustava
- npr. čvor za mapiranje komandi joysticka

Pokretanje:

```
ros2 run <package_name> <node_name>
```



ROS2 čvorovi - primjer

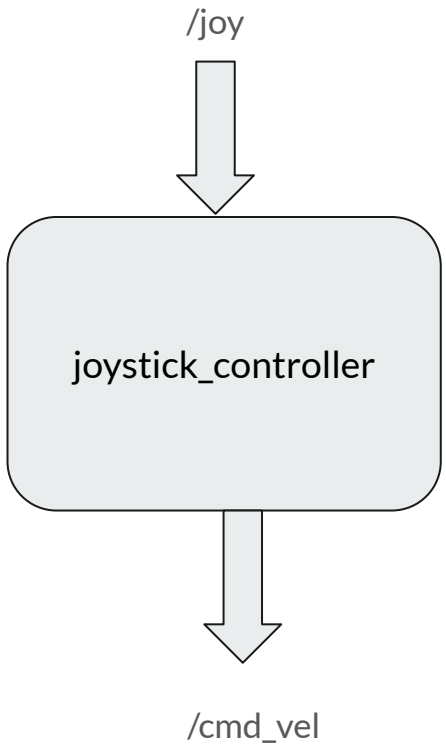
```python
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Joy
from geometry_msgs.msg import Twist

class JoystickController(Node):
    def __init__(self):
        super().__init__('joystick_controller')
        self.publisher = self.create_publisher(Twist, '/cmd_vel', 10)
        self.subscription = self.create_subscription(Joy, '/joy', self.joy_callback,
10)
        self.max_linear_speed = 1.0
        self.max_angular_speed = 2.0
        self.get_logger().info('Joystick controller started!')

    def joy_callback(self, msg):
        velocity = Twist()
        velocity.linear.x = msg.axes[1] * self.max_linear_speed
        velocity.angular.z = msg.axes[0] * self.max_angular_speed
        self.publisher.publish(velocity)

def main(args=None):
    rclpy.init(args=args)
    node = JoystickController()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

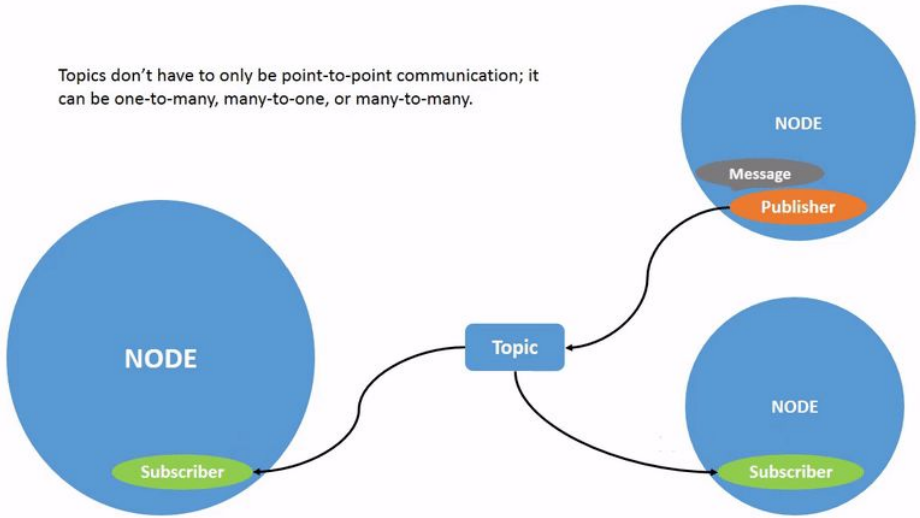
if __name__ == '__main__':
    main()
```



ROS2 teme

- Vrsta komunikacije između ROS2 čvorova - ROS2 *interface*
- Kontinuirani *stream* podataka
- *publisher* i *subscriber*
- Moguće koristiti standardne ROS2 poruke ili IZNIMNO definirati vlastitu

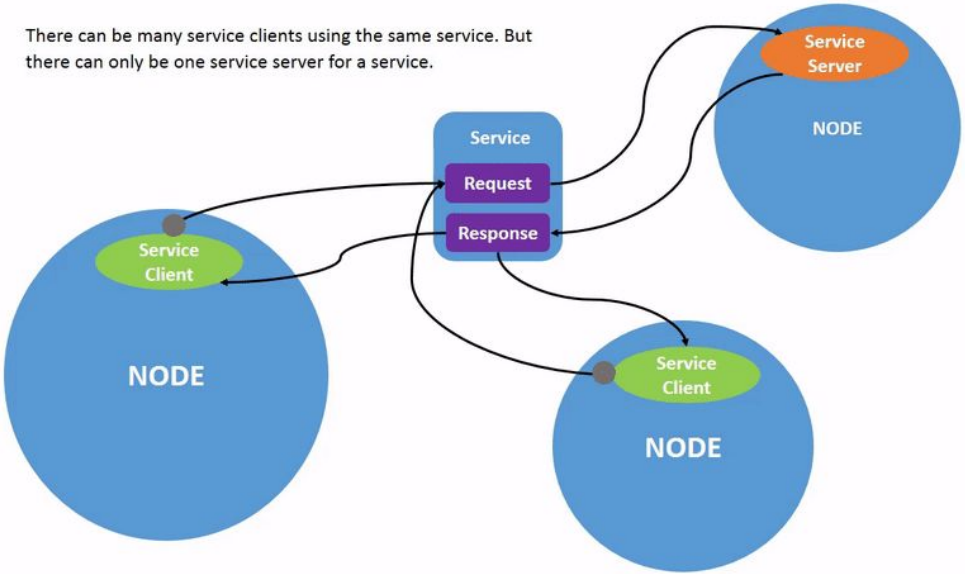
ros2 topic list



ROS2 servisi

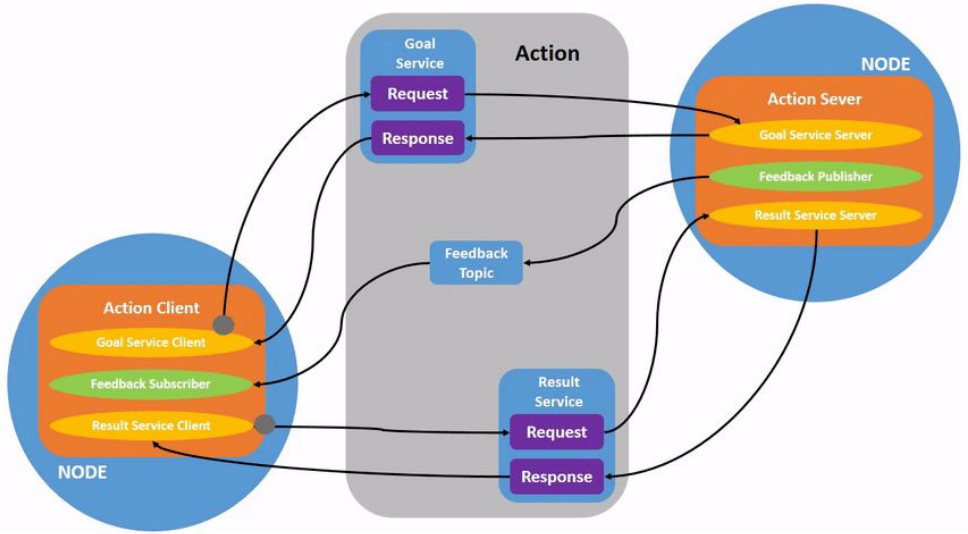
- Vrsta komunikacije između ROS2 čvorova
- *Request-response* oblik komunikacije
- *Server* i *client*
- Moguće koristiti standardne ROS2 service ili definirati vlastite (češće u praksi)

ros2 service list



ROS2 akcije

- Vrsta komunikacije između ROS2 čvorova
- *Request-feedback-delayed response* oblik komunikacije
- *Action server* i *client*
- Moguće koristiti standardne ROS2 akcije ili definirati vlastite (češće u praksi)
- Vrlo korisno u izvršavanju autonomnih radnji



ROS2 parametri

- Služe za parametrizaciju ros2 čvorova
- Pohranjeni u .yaml datoteci
- Svaki čvor ima svoju listu parametara
- Podržani tipovi podataka:
  - integeri
  - floatovi
  - boolovi
  - stringovi
  - liste

ROS2 launchfile

- Služe za pokretanje više ROS2 čvorova istovremeno
- U ROS2 to su ili python datoteke ili xml datoteke
- Moguće je kontrolirati i sekvencu paljenja čvorova, parametriziranje, remapiranje topica, namespaceove…

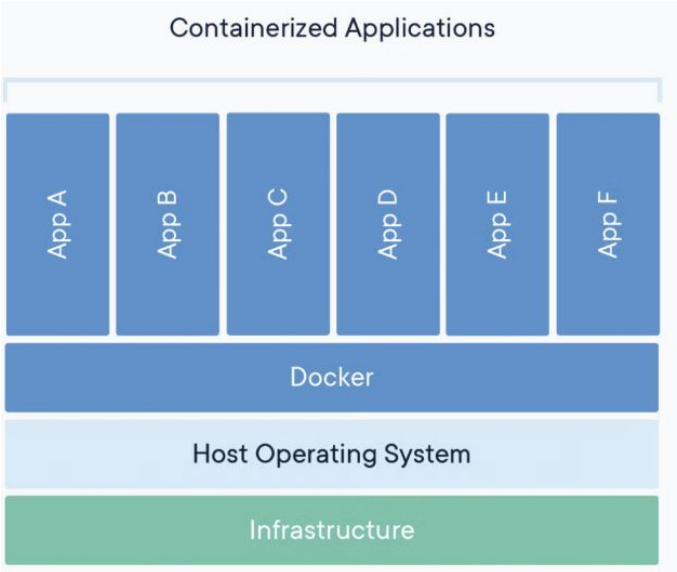
```
ros2 launch <package_name> <launch_file_name>
```

Više informacija o ROS2:

https://docs.ros.org/en/jazzy/index.html

Docker

- Omogućava izvršavanje aplikacija unutar izoliranog sustava - docker kontenjera
- Rješava jedan od najzastupljenijih problema softvera - “Ali radi na mojem računalu…”
- Portabilnost, kompatibilnost sa starijim verzijama ROS-a, mogućnost korištenja starijih paketa…
- Bitni pojmovi:
  - Dockerfile
  - Docker image
  - Docker container
  - Docker engine
  - Docker compose



Više informacija o Docker-u:

https://docs.docker.com/engine/

Git(hub)

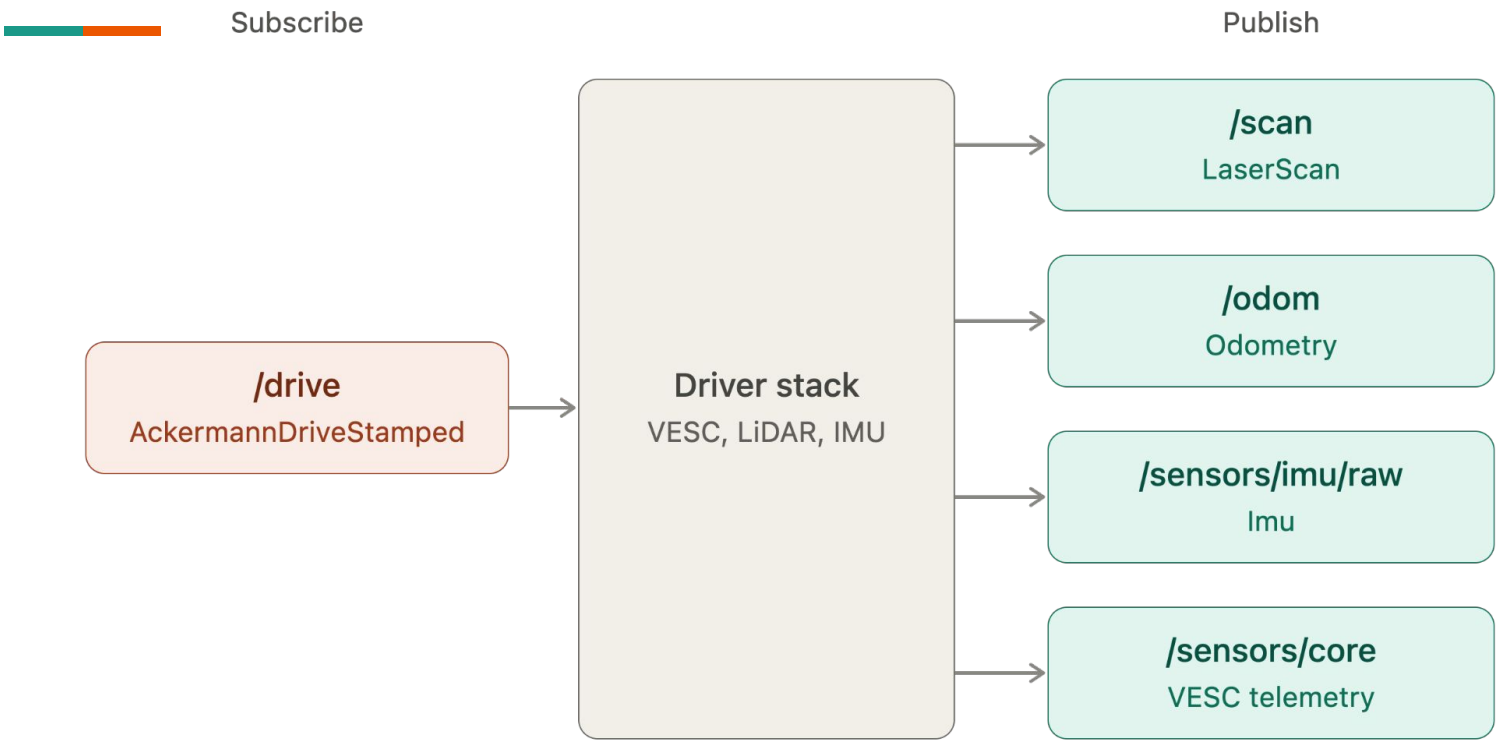
- Služi za praćenje softverskog razvoja
- Verzioniranje koda - omogućuje “spremanje” verzija koda koje rade
- Praćenje promjena u kodu
- Kolaboracija
- Izolira razvoj novih značajki od koda koji radi

Više informacija o Git-u:

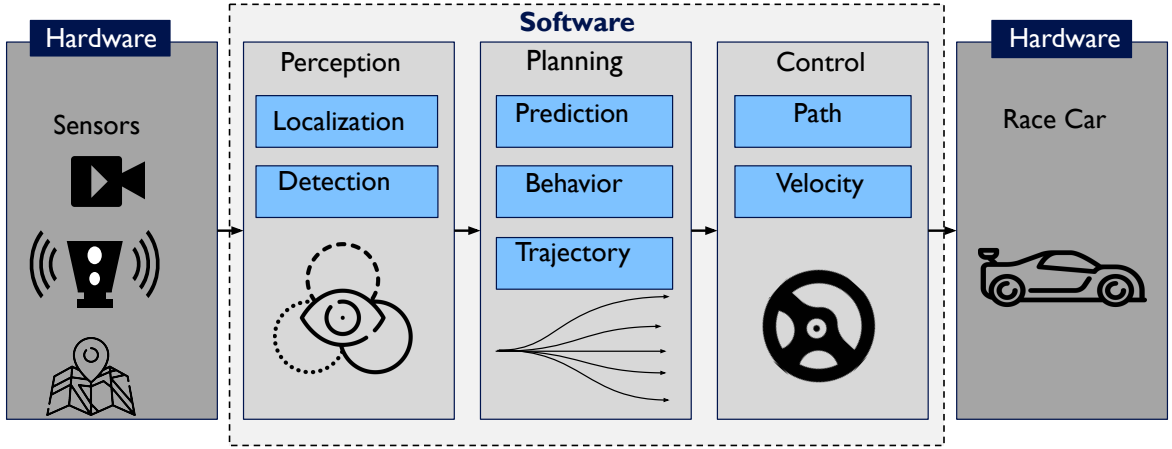
https://docs.github.com/en/get-started/using-github/hello-world

Softverska arhitektura vozila

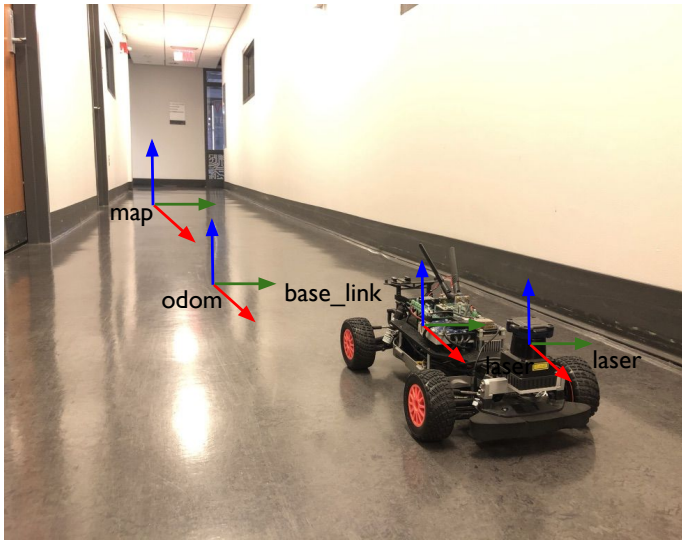
- Ključni topici: /scan (LiDAR), /odom (odometrija), /drive (AckermannDriveStamped – brzina + kut)
- Mux: prioritet joystick > autonomija
- Workflow: razvoj na laptopu → GitHub → pull i build na autu
- /drive: Topic for autonomous navigation, uses AckermannDriveStamped messages.


Subscribe
Publish
/drive
AckermannDriveStamped
Driver stack
VESC, LiDAR, IMU
/scan
LaserScan
/odom
Odometry
/sensors/imu/raw
Imu
/sensors/core
VESC telemetry

Autonomy pipeline



Koordinatni sustavi vozila



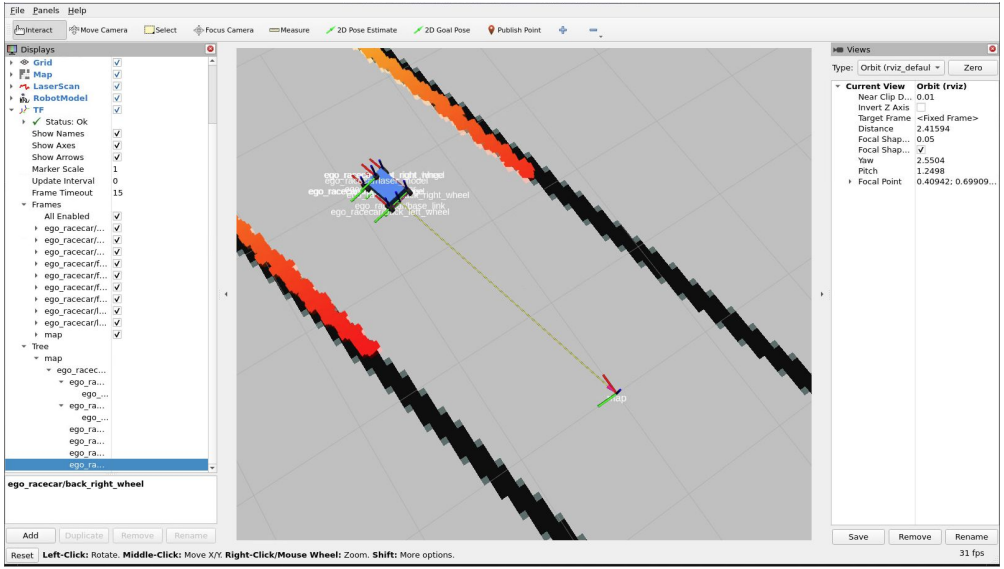
F1tenth gym

- 2D simulator napravljen u Pythonu
- Ubrzano izvršavanje (30x brže od stvarnog vremena)
- Realistična simulacija kinematike i dinamike vozila
- Realistična simulacija kolizije s objektima
- Odometrija i laser scan podaci

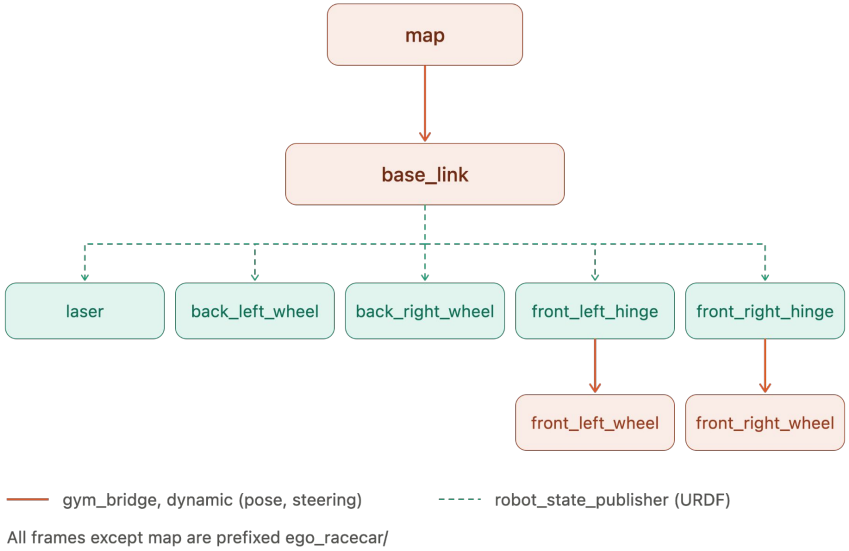


Koordinatni sustavi u simulatoru

- Simulator pretpostavlja da nema drifta

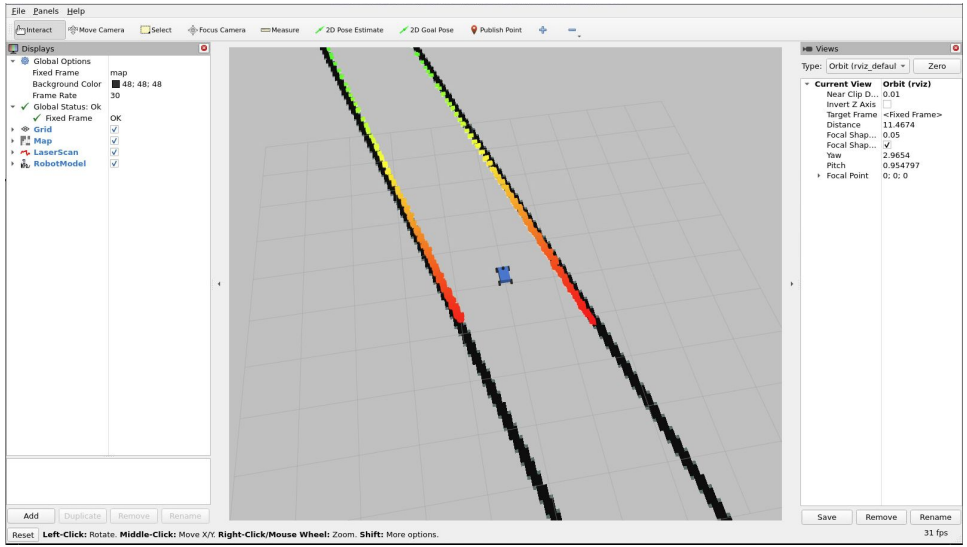


Koordinatni sustavi u simulatoru



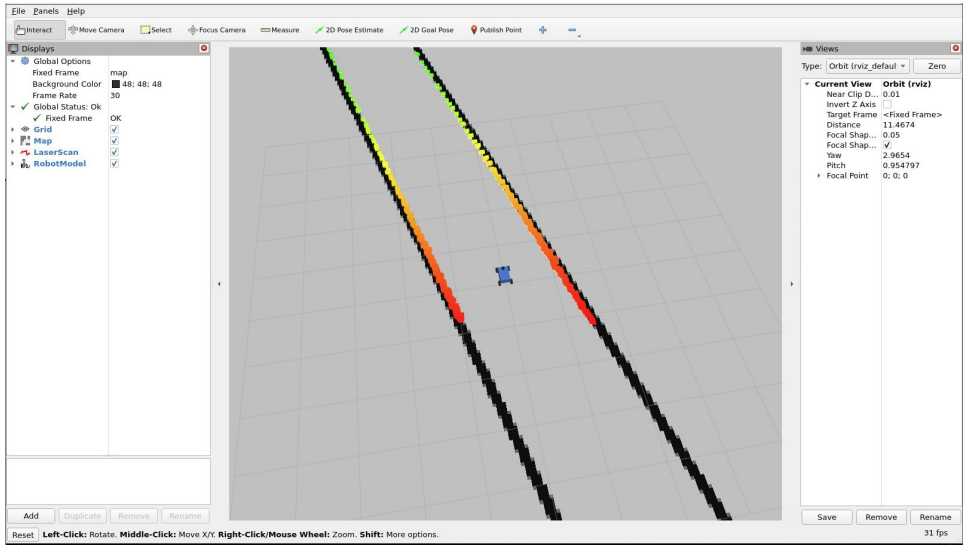
F1tenth gym

```
ros2 launch f1tenth_gym_ros \
gym_bridge_launch.py
```

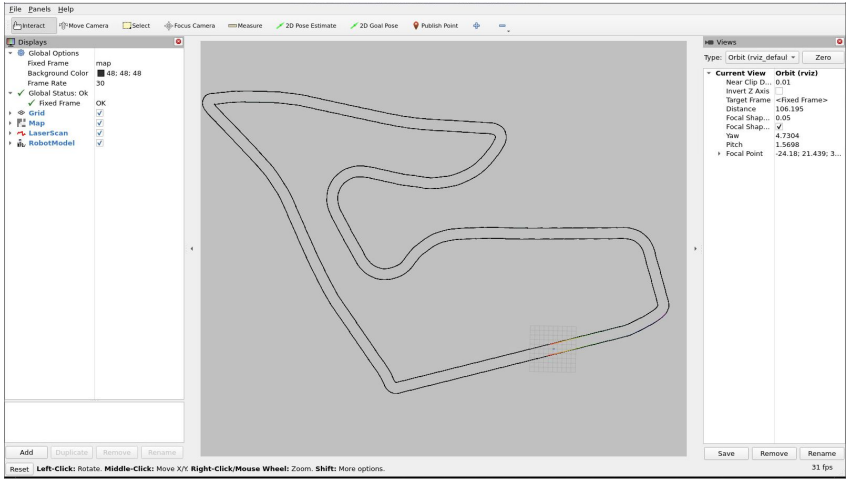


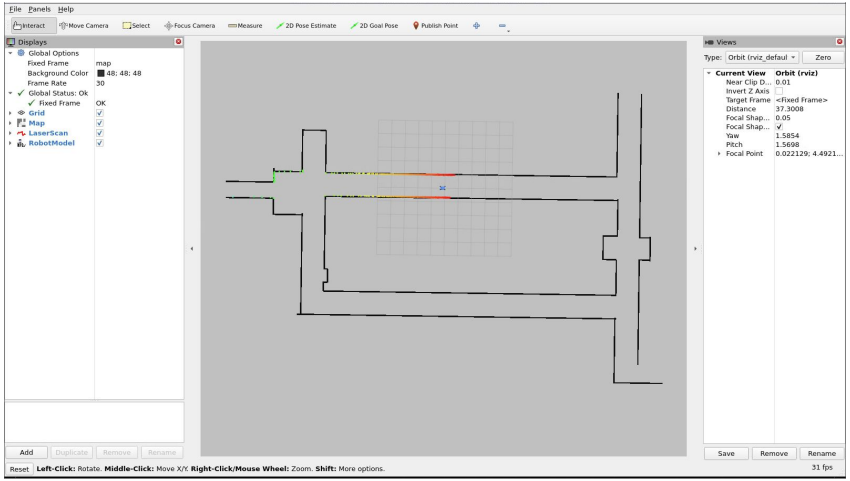
F1tenth gym

```
ros2 run teleop_twist_keyboard \
teleop_twist_keyboard
```



F1tenth gym mape

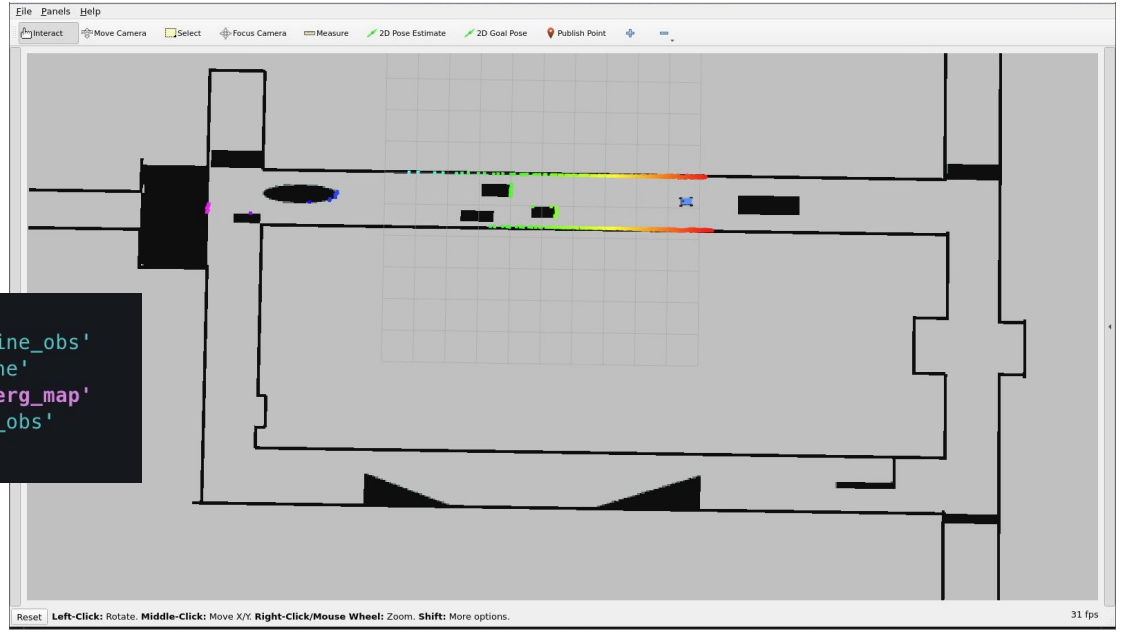




F1tenth gym: custom mape

- Potrebno urediti datoteku sim.yaml
-

```
# map parameters
#  map_path: '/sim_ws/src/f1tenth_gym_ros/maps/levine_obs'
# map_path: '/sim_ws/src/f1tenth_gym_ros/maps/levine'
map_path: '/sim_ws/src/f1tenth_gym_ros/maps/Spielberg_map'
# map_path: '/sim_ws/src/student/gap_follow/levine_obs'
map_img_ext: '.png'
```



F1tenth gym: Opponent

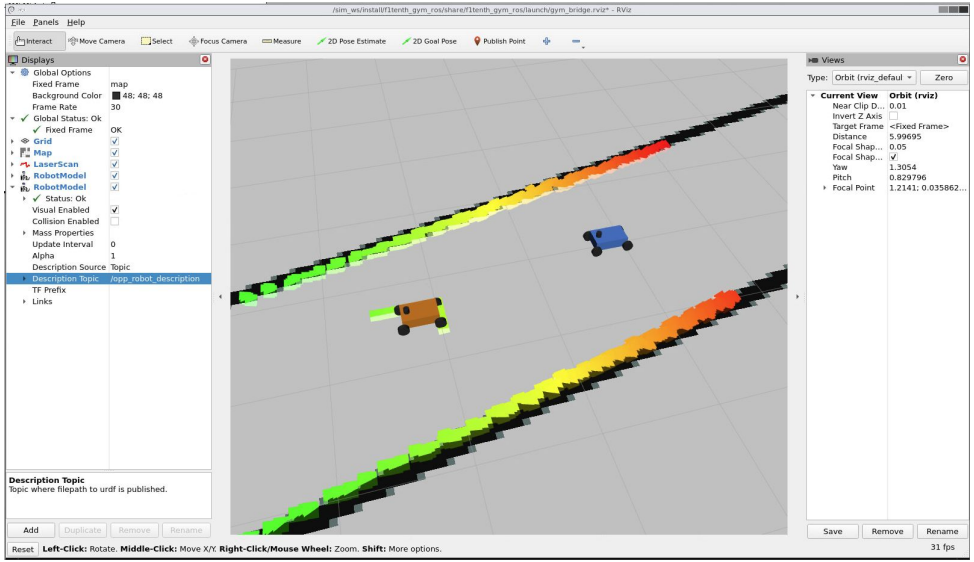
- Potrebno urediti datoteku sim.yaml

```
sim_ws/src/f1tenth_gym_ros/config/sim
.yaml
```

```yaml
# opponent parameters
num_agent: 2

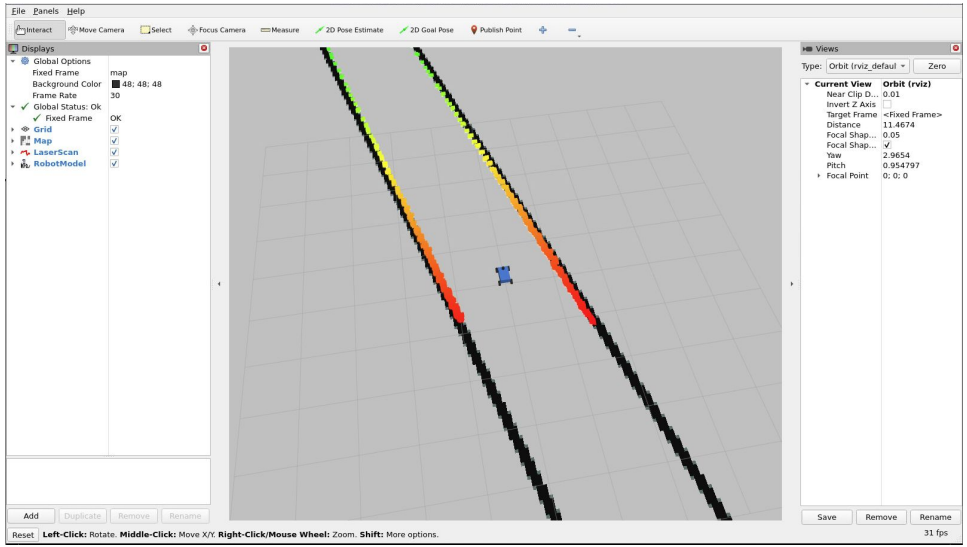
# ego starting pose on map
sx: 0.0
sy: 0.0
stheta: 0.0

# opp starting pose on map
sx1: 2.0
sy1: 0.5
```



Wall follow primjer

```
ros2 run wall_follow wall_follow
```



Struktura vježbi po tjednima

Tjedan	Tema vježbe
9.11. - 13.11.	Modeliranje i control
16.11. - 20.11.	Reaktivne metode
30.11. - 4.12.	Lokalizacija i mapiranje
7.12. - 11.12.	Planiranje putanje i slijeđenje putanje
TBD	Raceday 1 - simulator
TBD	Raceday 2 - uživo

Format natjecanja - simulator

- Time trial
- Ukupno dvije staze, jedna unaprijed poznata, druga zadana na dan natjecanja
- Vremena najbržih krugova na jednoj i drugoj stazi se zbrajaju
- Priprema - 2 tjedna, unaprijed poznata staza zadana u tjednu 30.11. - 4.12.
- Svi se natječu na istom računalu - portabilan kod!
- Određeno vrijeme pripreme za nepoznatu stazu
- Određeno vrijeme kvalifikacijske sesije
- Vremenski penali za kolizije
- TOP 6 vremena sudjeluje u raceday 2

Format natjecanja - uživo

- Time trial
- Staza nepoznata do dana natjecanja
- Priprema - traje od raceday 1 do raceday 2
- Organizirani termini za testiranje algoritama na pravom robotu u CRTI
- Na dan natjecanja:
  - priprema - 30 min, po 2 paralelno na stazi
  - utrka - 10 min (po sudioniku), najbrži krug
- Ostala pravila po: https://roboracer.ai/rules